

Artigo: Software Architecture: A Roadmap

David Garlan apresenta neste artigo uma visão abrangente do estado da arquitetura de software no final da década de 1990, traçando sua evolução desde práticas informais até o reconhecimento como disciplina de engenharia. O texto funciona como um mapa que localiza onde a área estava naquele momento e especula sobre para onde precisaria evoluir nos anos seguintes.

A importância de uma boa arquitetura de software é estabelecida logo no início como crítica para o sucesso de qualquer sistema complexo. Garlan argumenta que a arquitetura funciona como ponte entre requisitos e implementação, fornecendo uma descrição abstrata do sistema que expõe certas propriedades enquanto oculta outras. Um exemplo usado é o de uma aplicação de processamento de sinais estruturada como uma rede de fluxo de dados, onde arquitetos podem raciocinar sobre gargalos, requisitos de recursos e escalabilidade usando estimativas de fluxos de entrada, custos computacionais e capacidades de buffer. Essa abstração permite que designers entendam sistemas grandes de forma intelectualmente tratável, sugerem um blueprint para construção e facilitam análises de alta nível.

O trabalho identifica seis papéis principais que a arquitetura desempenha no desenvolvimento de software. Primeiro, a arquitetura simplifica compreensão de sistemas complexos, apresentando-os em um nível de abstração apropriado onde decisões de design de alto nível tornam-se inteligíveis. Além disso, descrições arquiteturais bem feitas expõem as restrições de alto nível do design e a fundamentação para escolhas específicas. Segundo, a arquitetura suporta reuse em múltiplos níveis, não apenas de componentes individuais, mas também de frameworks e arquiteturas de domínios específicos. Terceiro, as descrições arquiteturais fornecem um blueprint parcial para desenvolvimento ao indicar componentes principais e dependências entre eles, claramente identificando interfaces internas maiores do sistema. Quarto, a arquitetura expõe dimensões ao longo das quais um sistema deve evoluir, permitindo que mantenedores entendam melhor as ramificações de mudanças ao distinguir claramente entre componentes e mecanismos de interação. Quinto, descrições arquiteturais abrem oportunidades para análises como verificação de consistência e conformidade a atributos de qualidade. Sexto, a experiência industrial mostrou que alcançar uma arquitetura viável é um marco crítico em processos de desenvolvimento, com avaliações críticas levando a compreensão muito mais clara de requisitos, estratégias de implementação e riscos potenciais.

A progressão histórica que Garlan descreve é valiosa. Há uma década, a arquitetura era em grande medida um assunto ad hoc. Descrições se baseavam em diagramas informais de caixas e linhas raramente mantidos após a construção, e escolhas arquiteturais eram feitas de forma idiossincrática pela adaptação de designs anteriores. Bons arquitetos aprendiam seu ofício por experiência dura em domínios específicos mas eram incapazes de ensinar a outros o que sabiam. Não era possível analisar descrições arquiteturais para

consistência ou inferir propriedades não triviais. Contudo, apesar dessa informalidade, descrições arquiteturais eram centrais ao design de sistemas. Conforme as pessoas passaram a reconhecer o papel crítico que o design arquitetural desempenha, começaram a reconhecer a necessidade de uma abordagem mais disciplinada. Autores iniciais começaram a observar princípios unificadores, a convocar a arquitetura como campo necessitando atenção, e a estabelecer um vocabulário de trabalho.

O artigo então se volta para o futuro, identificando três tendências significativas que impactariam a arquitetura. Primeiro, o balanço dinâmico entre construir versus comprar estava se transformando. Pressões econômicas para reduzir time-to-market tornavam cada vez mais comum o desenvolvimento de sistemas predominantemente a partir de componentes externos, com código escrito internamente sendo principalmente "cola" para integração. Isso elevava a importância de padrões e frameworks compatíveis para reduzir problemas de incompatibilidade arquitetural. Segundo, havia uma transição clara de modelos computacionais centrados em PCs para modelos centrados em redes. Historicamente, sistemas eram desenvolvidos como sistemas fechados sob controle de instituições individuais, mas redes como a Internet apresentavam novos desafios: sistemas com controle descentralizado, com recursos independentemente desenvolvidos, suportados e até transientes, precisariam de arquiteturas que escalassem à variabilidade e tamanho da Internet, lidar com coligações formadas dinamicamente de recursos autônomos, e permitir composição por usuários finais sem profunda expertise técnica. Terceiro, computação pervasiva com dispositivos heterogêneos em proliferação criaria ambientes com centenas ou milhares de dispositivos exigindo atenção especial a uso de recursos, flexibilidade para reconfigurações dinâmicas, e gerenciamento muito mais automatizado.

O artigo também é honesto sobre a complexidade em balancear ideais. Garlan reconhece que embora estilos arquiteturais uniformes sejam valiosos, realidades da construção de software frequentemente forçam composição de partes não arquitetadas uniformemente, como quando se combina um banco de dados de um fornecedor com middleware de outro e interface de usuário de um terceiro. Isso o leva a discussão de estratégias para bridging mismatches e do reconhecimento de que essas técnicas ainda estavam pouco compreendidas.